

PARCELWALA · BACKEND SPECIFICATION

Smart Shifting

API changes for the new “what are you sending” booking flow

For Backend developer

Scope 2 modified APIs, 1 new API, 1 read endpoint

Compatibility Fully additive — nothing in the current parcel flow changes

App status Built, tested and ready; works today without any of this

WHAT CHANGES

- 1 **GET /vehicles/types** — modify, add 4 fields
- 2 **GET /moving/items** — new endpoint + table + 70-row seed
- 3 **POST /bookings** — modify, add 9 fields + 1 table
- 4 **GET /orders/{id}** — modify, return the same 9

Summary

#	API	Type of change	Priority
1	GET /vehicles/types	MODIFY — add 4 fields	● Highest
2	GET /moving/items	NEW — 1 endpoint + 1 table	● High
3	POST /bookings	MODIFY — add 9 fields + 1 table	● High
4	GET /orders/{id}	MODIFY — return the same 9 fields	● Medium

If you do only one thing, do #1. It is four columns and it is the difference between an exact vehicle recommendation and an educated guess.

1. GET /vehicles/types — MODIFY

What to add

Four fields on each vehicle. All optional.

Field	Type	Example	Meaning
capacity_cft	number	80	Usable loading volume in cubic feet. The important one.
deck_length_ft	number	7.0	Loading deck length
deck_width_ft	number	4.5	Loading deck width
deck_height_ft	number	4.5	Height goods can be stacked to (not side-rail height)

Response — before and after

```
// BEFORE (unchanged fields)
{
  "vehicle_type_id": 3,
  "name": "Tata Ace",
  "max_capacity_kg": 750,
  "capacity": "750 kg",
  "is_available": true
}

// AFTER — the 4 new fields at the end
{
  "vehicle_type_id": 3,
```

```
"name": "Tata Ace",
"max_capacity_kg": 750,
"capacity": "750 kg",
"is_available": true,

"capacity_cft": 80,
"deck_length_ft": 7.0,
"deck_width_ft": 4.5,
"deck_height_ft": 4.5
}
```

Database

```
ALTER TABLE vehicle_types
  ADD COLUMN capacity_cft DECIMAL(7,2) NULL,
  ADD COLUMN deck_length_ft DECIMAL(5,2) NULL,
  ADD COLUMN deck_width_ft DECIMAL(5,2) NULL,
  ADD COLUMN deck_height_ft DECIMAL(5,2) NULL;
```

Why it matters

Household goods **run out of space long before they run out of weight**. A Tata Ace is rated for 750 kg but holds only ~80 cubic feet — two sofas and a wardrobe fill it completely at **under 150 kg**.

Sizing on `max_capacity_kg` alone therefore recommends vehicles the goods physically cannot fit into. Without `capacity_cft`, the app has to guess your fleet's volumes from the vehicle name.

Values to start with

Seed these, then correct from real trips.

Vehicle	max_capacity_kg	capacity_cft	deck L × W × H (ft)
Bike / Scooter	20	3	1.5 × 1.5 × 1.5
E-loader	350	30	4 × 3 × 3
Auto / 3-Wheeler	500	45	5 × 3.5 × 3.5
Tata Ace	750	80	7 × 4.5 × 4.5
Pickup / Bolero	1250	130	8 × 5 × 4.5
Tempo / 407	2500	250	9.5 × 5.5 × 6
Canter 14 ft	3500	400	14 × 6 × 6

Vehicle	max_capacity_kg	capacity_cft	deck L × W × H (ft)
Truck 17 ft	5000	550	17 × 6 × 6.5
Truck 19 ft / Container	7000	700	19 × 7 × 7

Do not pre-discount `deck_height_ft` . The app already reduces the deck box by 15%, because nothing is packed to the very top.

Always send `is_available`

If this field is **missing** from the response, the app receives `false` — not `true` . (Gson creates objects in a way that skips Kotlin's default values.)

Smart Shifting is the only screen that reads this flag, so if it ever stops being sent, **that one flow would say "no vehicles available" for a perfectly healthy fleet** and nothing else in the app would look wrong. The app now defends against it, but please always send it explicitly.

The same applies to every boolean in section 2.

2. GET /moving/items — NEW

The list of things a customer can say they are sending. **This is a new endpoint and a new table.**

Endpoint

```
GET /moving/items
Authorization: Bearer <token>
```

No parameters. Returns the whole catalog (~70 rows, ~25 KB). The app caches it for 1 hour.

Response

```
{
  "success": true,
  "message": null,
  "data": [
    {
      "item_id": "sofa_3",          // stable id – bookings reference this
      "name": "Sofa",
      "category": "furniture",
      "icon": "🛋️",                // single emoji
      "volume_cft": 35.0,         // space on a loaded vehicle → drives sizing
      "weight_kg": 45.0,
      "length_ft": 6.5,          // shown to customer as
      "width_ft": 3.0,           //   "approx 6.5 × 3 × 3 ft"
      "height_ft": 3.0,
      "is_bulky": true,
      "is_fragile": false,
      "is_preset": false,
      "hint": "3 seater"
    }
  ]
}
```

Return `WHERE is_active = 1 ORDER BY sort_order`.

Fields

Field	Type	Required	Notes
<code>item_id</code>	string	☒	Stable forever. Bookings reference it.
<code>name</code>	string	☒	
<code>category</code>	enum	☒	<code>food</code> <code>small_items</code> <code>furniture</code> <code>appliances</code> <code>boxes</code> <code>full_house</code> <code>business</code>
<code>icon</code>	string	☒	One emoji
<code>volume_cft</code>	number	☒	Drives the vehicle choice — read the warning below
<code>weight_kg</code>	number	☒	Per unit
<code>length_ft</code>	number	☐	Display only
<code>width_ft</code>	number	☐	Display only
<code>height_ft</code>	number	☐	Display only
<code>is_bulky</code>	bool	☐	Needs two people / a wide doorway
<code>is_fragile</code>	bool	☐	Food, glass, electronics
<code>is_preset</code>	bool	☐	Whole-home preset (1 BHK etc.)
<code>hint</code>	string	☐	<code>"3 seater"</code> , <code>"Double door"</code>

`volume_cft` is NOT length × width × height

The most important rule in this document.

- `volume_cft` = the space the item takes **on a loaded vehicle**. A dining table travels on its side with the chairs stacked into the gap, so it is far less than its box.
- `length_ft` / `width_ft` / `height_ft` = so the customer can recognise the item. **Never multiplied to produce a volume.**

Rule to check against: for any item over ~3 cu.ft, `volume_cft` must be **less than** `length × width × height`. An object cannot take more room than its own bounding box. (Small items are the exception — a laptop needs a padded bag.)

Three more rules of thumb:

- Err generous.** A vehicle one size too big costs a few hundred rupees. One size too small costs the customer the whole day, and costs you the trip.
- `is_bulky` **means awkward, not large.** It adds packing space *and* removes the "any bulky items?" question from the customer's flow, because the app then already knows.
- `is_preset` **is whole homes only.** These replace the customer's item list rather than adding to it.

Table

```
CREATE TABLE moving_items (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  item_id     VARCHAR(64) NOT NULL UNIQUE,
  name        VARCHAR(128) NOT NULL,
  category    VARCHAR(32) NOT NULL,
  icon        VARCHAR(16) NOT NULL,
  volume_cft  DECIMAL(7,2) NOT NULL,
  weight_kg   DECIMAL(7,2) NOT NULL,
  length_ft   DECIMAL(5,2) NULL,
  width_ft    DECIMAL(5,2) NULL,
  height_ft   DECIMAL(5,2) NULL,
  is_bulky    TINYINT(1) NOT NULL DEFAULT 0,
  is_fragile  TINYINT(1) NOT NULL DEFAULT 0,
  is_preset   TINYINT(1) NOT NULL DEFAULT 0,
  hint        VARCHAR(128) NULL,
  sort_order  INT NOT NULL DEFAULT 0,
  is_active   TINYINT(1) NOT NULL DEFAULT 1,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  INDEX idx_category_active (category, is_active),
  INDEX idx_sort (sort_order)
);
```

Never delete a row — set `is_active = 0`. Bookings reference `item_id`, and a deleted row orphans those records.

Seed data — 70 items, ready to run

```
INSERT INTO moving_items
  (item_id, name, category, icon, volume_cft, weight_kg, length_ft, width_ft,
   height_ft, is_bulky, is_fragile, is_preset, hint, sort_order, is_active)
VALUES
-- FOOD (9)
('food_tiffin', 'Tiffin / lunch box', 'food', '🍱', 0.4, 1.5, 1.0, 0.8, 0.8, 0, 1, 0, 'Home food', 1, 1),
('food_meal', 'Packed meal', 'food', '🍲', 0.3, 1.0, 1.0, 0.8, 0.5, 0, 1, 0, 'Restaurant order', 2, 1),
('food_cake', 'Cake', 'food', '🍰', 1.2, 2.0, 1.0, 1.0, 1.0, 0, 1, 0, 'Boxed, keep flat', 3, 1),
('food_sweets', 'Sweets / mithai box', 'food', '🍬', 0.6, 2.0, 1.0, 1.0, 0.5, 0, 1, 0, NULL, 4, 1),
('food_groceries', 'Grocery bag', 'food', '🛒', 1.5, 8.0, 1.2, 1.0, 1.2, 0, 0, 0, NULL, 5, 1),
('food_vegetables', 'Vegetables / fruit crate', 'food', '🥬', 3.0, 15.0, 2.0, 1.3, 1.0, 0, 0, 0, NULL, 6, 1),
('food_water_can', 'Water can', 'food', '💧', 1.5, 20.0, 1.0, 1.0, 1.5, 0, 0, 0, '20 litre', 7, 1),
('food_milk_crate', 'Milk / dairy crate', 'food', '🥛', 2.0, 18.0, 1.7, 1.2, 1.0, 0, 1, 0, NULL, 8, 1),
('food_catering', 'Catering vessel', 'food', '🍲', 6.0, 25.0, 2.0, 2.0, 1.5, 0, 1, 0, 'Large deggh', 9, 1),
```

```

-- SMALL ITEMS (10)

('small_documents', 'Documents / envelope', 'small_items', '📄', 0.2, 0.5, 1.2, 0.9, 0.2, 0, 0, 0, NULL, 10, 1),
('small_keys', 'Keys / small packet', 'small_items', '🔑', 0.1, 0.3, 0.5, 0.4, 0.3, 0, 0, 0, NULL, 11, 1),
('small_medicines', 'Medicines', 'small_items', '💊', 0.4, 1.0, 1.0, 0.8, 0.5, 0, 1, 0, NULL, 12, 1),
('small_laptop', 'Laptop / tablet', 'small_items', '💻', 0.8, 3.0, 1.3, 1.0, 0.4, 0, 1, 0, NULL, 13, 1),
('small_mobile', 'Mobile / gadget', 'small_items', '📱', 0.2, 0.5, 0.7, 0.5, 0.3, 0, 1, 0, NULL, 14, 1),
('small_gift', 'Gift / flowers', 'small_items', '🎁', 1.5, 3.0, 1.3, 1.0, 1.3, 0, 1, 0, NULL, 15, 1),
('small_clothes', 'Clothes packet', 'small_items', '👕', 1.5, 4.0, 1.5, 1.0, 0.8, 0, 0, 0, NULL, 16, 1),
('small_book_parcel', 'Books parcel', 'small_items', '📚', 1.5, 10.0, 1.3, 1.0, 1.0, 0, 0, 0, NULL, 17, 1),
('small_spare_part', 'Spare part / tool', 'small_items', '🔧', 1.5, 8.0, 1.5, 1.0, 0.8, 0, 0, 0, NULL, 18, 1),
('small_carton', 'Single carton', 'small_items', '📦', 3.0, 10.0, 1.7, 1.3, 1.3, 0, 0, 0, 'Standard courier box', 19, 1),

-- FURNITURE (16)

('sofa_3', 'Sofa', 'furniture', '🛋️', 35.0, 45.0, 6.5, 3.0, 3.0, 1, 0, 0, '3 seater', 20, 1),
('sofa_2', 'Sofa', 'furniture', '🛋️', 24.0, 32.0, 4.5, 3.0, 3.0, 1, 0, 0, '2 seater', 21, 1),
('sofa_1', 'L-shape sofa', 'furniture', '🛋️', 55.0, 70.0, 8.0, 6.0, 3.0, 1, 0, 0, 'Corner set', 22, 1),
('bed_double', 'Double bed', 'furniture', '🛏️', 30.0, 60.0, 6.5, 5.0, 2.5, 1, 0, 0, 'With headboard', 23, 1),
('bed_single', 'Single bed', 'furniture', '🛏️', 18.0, 35.0, 6.5, 3.0, 2.0, 1, 0, 0, NULL, 24, 1),
('mattress', 'Mattress', 'furniture', '🛏️', 14.0, 25.0, 6.5, 5.0, 0.7, 1, 0, 0, 'Queen / king', 25, 1),
('wardrobe', 'Wardrobe', 'furniture', '🚪', 38.0, 70.0, 4.0, 2.0, 6.5, 1, 0, 0, '2 door', 26, 1),
('almirah', 'Steel almirah', 'furniture', '🚪', 26.0, 80.0, 3.0, 1.7, 6.0, 1, 0, 0, NULL, 27, 1),
('dining_table', 'Dining table', 'furniture', '🍽️', 22.0, 40.0, 5.0, 3.0, 2.5, 1, 0, 0, 'Top only', 28, 1),
('chair', 'Chair', 'furniture', '🪑', 6.0, 7.0, 1.5, 1.5, 3.0, 0, 0, 0, NULL, 29, 1),
('study_table', 'Study table', 'furniture', '📖', 14.0, 25.0, 4.0, 2.0, 2.5, 0, 0, 0, NULL, 30, 1),
('bookshelf', 'Bookshelf', 'furniture', '📚', 16.0, 35.0, 3.0, 1.2, 5.0, 0, 0, 0, NULL, 31, 1),
('shoe Rack', 'Shoe rack', 'furniture', '👟', 9.0, 14.0, 2.5, 1.2, 3.0, 0, 0, 0, NULL, 32, 1),
('center_table', 'Center table', 'furniture', '🍷', 8.0, 15.0, 3.0, 2.0, 1.5, 0, 0, 0, NULL, 33, 1),
('tv_unit', 'TV unit', 'furniture', '📺', 10.0, 30.0, 4.0, 1.5, 2.0, 0, 0, 0, NULL, 34, 1),
('mirror', 'Mirror / dressing table', 'furniture', '🪞', 12.0, 22.0, 3.0, 1.5, 5.0, 1, 1, 0, NULL, 35, 1),

-- APPLIANCES (10)

('fridge_double', 'Refrigerator', 'appliances', '🧊', 28.0, 90.0, 2.5, 2.5, 6.0, 1, 0, 0, 'Double door', 36, 1),
('fridge_single', 'Refrigerator', 'appliances', '🧊', 16.0, 55.0, 2.0, 2.0, 5.0, 1, 0, 0, 'Single door', 37, 1),
('washing_machine', 'Washing machine', 'appliances', '🌀', 15.0, 65.0, 2.2, 2.2, 3.5, 1, 0, 0, NULL, 38, 1),
('ac_split', 'AC unit', 'appliances', '❄️', 12.0, 45.0, 3.5, 2.5, 2.0, 1, 0, 0, 'Split, both units', 39, 1),
('tv', 'Television', 'appliances', '📺', 8.0, 18.0, 4.5, 0.8, 3.0, 1, 1, 0, 'Boxed', 40, 1),
('microwave', 'Microwave / oven', 'appliances', '🍳', 3.5, 15.0, 2.0, 1.5, 1.3, 0, 0, 0, NULL, 41, 1),
('gas_stove', 'Gas stove + cylinder', 'appliances', '🔥', 6.0, 25.0, 2.0, 1.5, 2.0, 0, 0, 0, NULL, 42, 1),
('water_purifier', 'Water purifier', 'appliances', '💧', 2.5, 12.0, 1.3, 1.0, 1.7, 0, 1, 0, NULL, 43, 1),
('cooler', 'Air cooler', 'appliances', '💨', 14.0, 20.0, 2.0, 2.0, 3.5, 1, 0, 0, NULL, 44, 1),
('geyser', 'Geyser', 'appliances', '🔥', 4.0, 14.0, 1.5, 1.5, 2.0, 0, 0, 0, NULL, 45, 1),

-- BOXES & BAGS (8)

('box_small', 'Box', 'boxes', '📦', 2.0, 8.0, 1.3, 1.3, 1.3, 0, 0, 0, 'Small', 46, 1),
('box_medium', 'Box', 'boxes', '📦', 4.0, 15.0, 1.7, 1.7, 1.7, 0, 0, 0, 'Medium', 47, 1),
('box_large', 'Box', 'boxes', '📦', 7.0, 22.0, 2.0, 2.0, 2.0, 0, 0, 0, 'Large', 48, 1),
('suitcase', 'Suitcase', 'boxes', '🧳', 3.2, 18.0, 2.3, 1.5, 1.0, 0, 0, 0, NULL, 49, 1),
('bag', 'Bag / sack', 'boxes', '🛍️', 3.0, 12.0, 1.7, 1.2, 1.5, 0, 0, 0, NULL, 50, 1),
('trunk', 'Trunk', 'boxes', '🚪', 6.0, 25.0, 3.0, 1.7, 1.5, 0, 0, 0, NULL, 51, 1),
('plants', 'Plant / pot', 'boxes', '🌱', 3.0, 10.0, 1.3, 1.3, 2.5, 0, 1, 0, NULL, 52, 1),

```

```

('cycle', 'Bicycle', 'boxes', '🚲', 10.0, 15.0, 6.0, 1.5, 3.5, 1, 0, 0, NULL, 53, 1),
-- FULL HOUSE presets (5)
('home_1rk', '1 RK', 'full_house', '🏠', 110.0, 320.0, NULL, NULL, NULL, 1, 0, 1, 'Studio / single room', 54, 1),
('home_1bhk', '1 BHK', 'full_house', '🏠', 175.0, 520.0, NULL, NULL, NULL, 1, 0, 1, 'Typical 1 bedroom', 55, 1),
('home_2bhk', '2 BHK', 'full_house', '🏠', 300.0, 900.0, NULL, NULL, NULL, 1, 0, 1, 'Typical 2 bedroom', 56, 1),
('home_3bhk', '3 BHK', 'full_house', '🏠', 450.0, 1350.0, NULL, NULL, NULL, 1, 0, 1, 'Typical 3 bedroom', 57, 1),
('home_4bhk', '4 BHK / villa', 'full_house', '🏠', 620.0, 1900.0, NULL, NULL, NULL, 1, 0, 1, 'Large home', 58, 1),
-- BUSINESS GOODS (12)
('office_desk', 'Office desk', 'business', '🖨️', 20.0, 35.0, 5.0, 2.5, 2.5, 1, 0, 0, NULL, 59, 1),
('office_chair', 'Office chair', 'business', '🪑', 8.0, 12.0, 2.0, 2.0, 3.5, 0, 0, 0, NULL, 60, 1),
('filing_cabinet', 'Filing cabinet', 'business', '📁', 11.0, 45.0, 1.5, 2.0, 4.0, 1, 0, 0, NULL, 61, 1),
('computer', 'Computer / monitor', 'business', '💻', 3.0, 12.0, 2.0, 1.0, 1.7, 0, 1, 0, NULL, 62, 1),
('printer', 'Printer', 'business', '🖨️', 5.0, 20.0, 2.0, 1.7, 1.5, 0, 1, 0, NULL, 63, 1),
('server_rack', 'Server rack', 'business', '🖥️', 24.0, 110.0, 2.5, 3.0, 6.0, 1, 1, 0, NULL, 64, 1),
('display_rack', 'Display rack', 'business', '🖥️', 22.0, 40.0, 4.0, 1.5, 6.0, 1, 0, 0, NULL, 65, 1),
('counter', 'Counter table', 'business', '📐', 25.0, 55.0, 6.0, 2.0, 3.5, 1, 0, 0, NULL, 66, 1),
('stock_carton', 'Stock carton', 'business', '📦', 4.0, 18.0, 1.7, 1.7, 1.7, 0, 0, 0, NULL, 67, 1),
('sack', 'Sack / gunny bag', 'business', '📦', 3.0, 30.0, 2.5, 1.5, 1.0, 0, 0, 0, NULL, 68, 1),
('drum', 'Drum / barrel', 'business', '🥁', 9.0, 60.0, 2.0, 2.0, 3.0, 1, 0, 0, NULL, 69, 1),
('pallet', 'Pallet', 'business', '📦', 30.0, 250.0, 4.0, 4.0, 4.0, 1, 0, 0, NULL, 70, 1);

```

The catalog, readable

Same 70 rows as the SQL above. `volume_cft` is what sizes the vehicle; `l×w×h` is what the customer sees.

FOOD (9)

item_id	name	volume_cft	weight_kg	l×w×h (ft)	flags
food_tiffin	Tiffin / lunch box <i>Home food</i>	0.4	1.5	1×0.8×0.8	fragile
food_meal	Packed meal <i>Restaurant order</i>	0.3	1	1×0.8×0.5	fragile
food_cake	Cake <i>Boxed, keep flat</i>	1.2	2	1×1×1	fragile
food_sweets	Sweets / mithai box	0.6	2	1×1×0.5	fragile
food_groceries	Grocery bag	1.5	8	1.2×1×1.2	—
food_vegetables	Vegetables / fruit crate	3	15	2×1.3×1	—
food_water_can	Water can <i>20 litre</i>	1.5	20	1×1×1.5	—
food_milk_crate	Milk / dairy crate	2	18	1.7×1.2×1	fragile
food_catering	Catering vessel <i>Large deggh</i>	6	25	2×2×1.5	fragile

SMALL ITEMS (10)

item_id	name	volume_cft	weight_kg	l×w×h (ft)	flags
small_documents	Documents / envelope	0.2	0.5	1.2×0.9×0.2	—
small_keys	Keys / small packet	0.1	0.3	0.5×0.4×0.3	—
small_medicines	Medicines	0.4	1	1×0.8×0.5	fragile
small_laptop	Laptop / tablet	0.8	3	1.3×1×0.4	fragile
small_mobile	Mobile / gadget	0.2	0.5	0.7×0.5×0.3	fragile
small_gift	Gift / flowers	1.5	3	1.3×1×1.3	fragile
small_clothes	Clothes packet	1.5	4	1.5×1×0.8	—
small_book_parcel	Books parcel	1.5	10	1.3×1×1	—
small_spare_part	Spare part / tool	1.5	8	1.5×1×0.8	—
small_carton	Single carton <i>Standard courier box</i>	3	10	1.7×1.3×1.3	—

FURNITURE (16)

item_id	name	volume_cft	weight_kg	l×w×h (ft)	flags
sofa_3	Sofa <i>3 seater</i>	35	45	6.5×3×3	bulky
sofa_2	Sofa <i>2 seater</i>	24	32	4.5×3×3	bulky
sofa_1	L-shape sofa <i>Corner set</i>	55	70	8×6×3	bulky
bed_double	Double bed <i>With headboard</i>	30	60	6.5×5×2.5	bulky
bed_single	Single bed	18	35	6.5×3×2	bulky
mattress	Mattress <i>Queen / king</i>	14	25	6.5×5×0.7	bulky
wardrobe	Wardrobe <i>2 door</i>	38	70	4×2×6.5	bulky
almirah	Steel almirah	26	80	3×1.7×6	bulky
dining_table	Dining table <i>Top only</i>	22	40	5×3×2.5	bulky
chair	Chair	6	7	1.5×1.5×3	—
study_table	Study table	14	25	4×2×2.5	—
bookshelf	Bookshelf	16	35	3×1.2×5	—
shoe_rack	Shoe rack	9	14	2.5×1.2×3	—
center_table	Center table	8	15	3×2×1.5	—

item_id	name	volume_cft	weight_kg	l×w×h (ft)	flags
tv_unit	TV unit	10	30	4×1.5×2	—
mirror	Mirror / dressing table	12	22	3×1.5×5	bulky, fragile

APPLIANCES (10)

item_id	name	volume_cft	weight_kg	l×w×h (ft)	flags
fridge_double	Refrigerator <i>Double door</i>	28	90	2.5×2.5×6	bulky
fridge_single	Refrigerator <i>Single door</i>	16	55	2×2×5	bulky
washing_machine	Washing machine	15	65	2.2×2.2×3.5	bulky
ac_split	AC unit <i>Split, both units</i>	12	45	3.5×2.5×2	bulky
tv	Television <i>Boxed</i>	8	18	4.5×0.8×3	bulky, fragile
microwave	Microwave / oven	3.5	15	2×1.5×1.3	—
gas_stove	Gas stove + cylinder	6	25	2×1.5×2	—
water_purifier	Water purifier	2.5	12	1.3×1×1.7	fragile
cooler	Air cooler	14	20	2×2×3.5	bulky
geyser	Geyser	4	14	1.5×1.5×2	—

BOXES & BAGS (8)

item_id	name	volume_cft	weight_kg	l×w×h (ft)	flags
box_small	Box <i>Small</i>	2	8	1.3×1.3×1.3	—
box_medium	Box <i>Medium</i>	4	15	1.7×1.7×1.7	—
box_large	Box <i>Large</i>	7	22	2×2×2	—
suitcase	Suitcase	3.2	18	2.3×1.5×1	—
bag	Bag / sack	3	12	1.7×1.2×1.5	—
trunk	Trunk	6	25	3×1.7×1.5	—
plants	Plant / pot	3	10	1.3×1.3×2.5	fragile
cycle	Bicycle	10	15	6×1.5×3.5	bulky

FULL HOUSE presets (5)

item_id	name	volume_cft	weight_kg	l×w×h (ft)	flags
home_1rk	1 RK <i>Studio / single room</i>	110	320	—	bulky, preset
home_1bhk	1 BHK <i>Typical 1 bedroom</i>	175	520	—	bulky, preset
home_2bhk	2 BHK <i>Typical 2 bedroom</i>	300	900	—	bulky, preset
home_3bhk	3 BHK <i>Typical 3 bedroom</i>	450	1350	—	bulky, preset
home_4bhk	4 BHK / villa <i>Large home</i>	620	1900	—	bulky, preset

BUSINESS GOODS (12)

item_id	name	volume_cft	weight_kg	l×w×h (ft)	flags
office_desk	Office desk	20	35	5×2.5×2.5	bulky
office_chair	Office chair	8	12	2×2×3.5	—
filing_cabinet	Filing cabinet	11	45	1.5×2×4	bulky
computer	Computer / monitor	3	12	2×1×1.7	fragile
printer	Printer	5	20	2×1.7×1.5	fragile
server_rack	Server rack	24	110	2.5×3×6	bulky, fragile
display_rack	Display rack	22	40	4×1.5×6	bulky
counter	Counter table	25	55	6×2×3.5	bulky
stock_carton	Stock carton	4	18	1.7×1.7×1.7	—
sack	Sack / gunny bag	3	30	2.5×1.5×1	—
drum	Drum / barrel	9	60	2×2×3	bulky
pallet	Pallet	30	250	4×4×4	bulky

3. POST /bookings — MODIFY

Store what the customer said they are sending.

What to add

Nine fields on the request. **All optional and all absent for a normal parcel booking**, so the existing flow is untouched.

Field	Type	Example	Meaning
booking_source	string	"moving"	Absent for a normal parcel booking
moving_items	array	see below	One row per item — the main thing to store
estimated_volume_cft	number	88.0	Total after packing allowance
load_size	string	"LARGE"	SMALL MEDIUM LARGE VERY_LARGE EXTRA_LARGE
has_bulky_items	bool	true	Needs two people / a wide door
has_fragile_items	bool	false	Food, glass, electronics
suggested_helpers	int	1	0 when the driver can manage alone
recommended_vehicle_type_id	int	4	What the app suggested, before any customer change
fits_in_one_trip	bool	true	⚠ see the warning below

Request

```
{
  // ...every existing field is unchanged...

  "booking_source": "moving",

  "moving_items": [
    {
      "item_id": "sofa_3",
      "name": "Sofa",
      "category": "furniture",
      "quantity": 1,
      "unit_volume_cft": 35.0,
      "unit_weight_kg": 45.0,
      "is_bulky": true,
      "is_fragile": false,
      "is_custom": false      // true = customer typed the item in themselves
    }
  ]
}
```

```

    },
    {
      "item_id": "box_medium",
      "name": "Box",
      "category": "boxes",
      "quantity": 8,
      "unit_volume_cft": 4.0,
      "unit_weight_kg": 15.0,
      "is_bulky": false,
      "is_fragile": false,
      "is_custom": false
    }
  ],

  "estimated_volume_cft": 88.0,
  "load_size": "LARGE",
  "has_bulky_items": true,
  "has_fragile_items": false,
  "suggested_helpers": 1,
  "recommended_vehicle_type_id": 4,
  "fits_in_one_trip": true
}

```

`goods_type_name` also carries a readable line for the driver, using fields you already have:

```
"House Shifting · Sofa x1, Box x8"
```



`fits_in_one_trip: false` **needs an operations action**

This is `false` when the load is bigger than anything in your fleet.

The customer is never told this. An earlier build showed "Needs 2+ trips" on screen; it appeared at the exact moment the customer was deciding whether to trust the estimate, read as the app refusing the job, and they left. The app now always recommends one vehicle and sends you the flag instead.

What to do: flag these bookings and call the customer to arrange a second vehicle **before a driver is dispatched**. It will be rare — usually 3 BHK and above.

```

SELECT * FROM bookings
WHERE booking_source = 'moving' AND fits_in_one_trip = 0
      AND status IN ('pending', 'searching')
ORDER BY created_at DESC;

```

Two notes on the data

Store `unit_volume_cft` and `unit_weight_kg` as sent — do not join them from `moving_items` at read time. If ops later corrects a sofa from 35 to 30 cft, a booking made last month must still show the numbers its recommendation was actually based on. Otherwise every historical report silently rewrites itself.

`recommended_vehicle_type_id` is the app's own suggestion, not the vehicle booked. Compare it with the booking's `vehicle_type_id` to see whether customers accept the recommendation — that comparison is the main signal for improving the catalog.

Database

```
ALTER TABLE bookings
  ADD COLUMN booking_source          VARCHAR(16) NULL,
  ADD COLUMN estimated_volume_cft    DECIMAL(8,2) NULL,
  ADD COLUMN load_size               VARCHAR(16) NULL,
  ADD COLUMN has_bulky_items         TINYINT(1) NULL,
  ADD COLUMN has_fragile_items       TINYINT(1) NULL,
  ADD COLUMN suggested_helpers       INT NULL,
  ADD COLUMN recommended_vehicle_type_id INT NULL,
  ADD COLUMN fits_in_one_trip        TINYINT(1) NULL,
  ADD INDEX idx_source (booking_source),
  ADD INDEX idx_needs_attention (booking_source, fits_in_one_trip);
```

```
CREATE TABLE booking_moving_items (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  booking_id  INT NOT NULL,
  item_id     VARCHAR(64) NOT NULL,
  name        VARCHAR(128) NOT NULL,
  category    VARCHAR(32) NOT NULL,
  quantity    INT NOT NULL,
  unit_volume_cft DECIMAL(7,2) NOT NULL,
  unit_weight_kg DECIMAL(7,2) NOT NULL,
  is_bulky     TINYINT(1) NOT NULL DEFAULT 0,
  is_fragile   TINYINT(1) NOT NULL DEFAULT 0,
  is_custom    TINYINT(1) NOT NULL DEFAULT 0,
  created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (booking_id) REFERENCES bookings(id) ON DELETE CASCADE,
  INDEX idx_booking (booking_id),
  INDEX idx_item (item_id)
);
```

No foreign key from `booking_moving_items.item_id` to `moving_items.item_id` — on purpose. Custom items (`is_custom = 1`) have ids the catalog has never seen, and a constraint would reject the whole booking.

4. GET /orders/{id} — MODIFY

Return the same block on order reads, so the **driver app** knows what it is collecting and **support** can answer "what did they book?".

```
{
  "booking_id": 1234,
  // ...existing fields...

  "booking_source": "moving",
  "load_size": "LARGE",
  "estimated_volume_cft": 88.0,
  "has_bulky_items": true,
  "has_fragile_items": false,
  "suggested_helpers": 1,
  "fits_in_one_trip": true,
  "moving_items": [ /* same shape as the request */ ]
}
```

Most useful for the driver app: `moving_items`, `has_bulky_items`, `has_fragile_items`, `suggested_helpers`. A driver who knows there is a fridge and a two-person lift *before* arriving is a driver who brings help.

5. Two reports worth adding later

No app change needed. Both pay for themselves quickly.

Is the app recommending the right vehicle?

```
SELECT v.name AS recommended, v2.name AS booked, COUNT(*) AS n
FROM bookings b
JOIN vehicle_types v ON v.id = b.recommended_vehicle_type_id
JOIN vehicle_types v2 ON v2.id = b.vehicle_type_id
WHERE b.booking_source = 'moving'
      AND b.recommended_vehicle_type_id <> b.vehicle_type_id
GROUP BY 1, 2 ORDER BY n DESC;
```

A consistent gap in one direction means the catalog volumes need adjusting, and `booking_moving_items` shows which items are involved.

What is missing from the catalog?

```
SELECT name, COUNT(*) AS times_typed
FROM booking_moving_items
WHERE is_custom = 1
GROUP BY name HAVING COUNT(*) > 3
ORDER BY times_typed DESC;
```

Every custom item is a customer telling you the catalog is missing something.

6. Order of work

Step	Task	Effort
1	Add 4 columns to <code>vehicle_types</code> , return them in <code>GET /vehicles/types</code>	~half a day
2	Create <code>moving_items</code> , run the seed, build <code>GET /moving/items</code>	~1 day
3	Add the booking columns + <code>booking_moving_items</code> , store on <code>POST /bookings</code>	~1 day
4	Return the block on <code>GET /orders/{id}</code>	~2 hours
5	Ops filter for <code>fits_in_one_trip = 0</code>	~2 hours

The app is already written against all of it. Nothing needs to change in the app as each step lands — the flow simply becomes more accurate.

Quick reference

Endpoint	New fields
GET /vehicles/types	capacity_cft , deck_length_ft , deck_width_ft , deck_height_ft
GET /moving/items	entire endpoint — 13 fields per item
POST /bookings	booking_source , moving_items[] , estimated_volume_cft , load_size , has_bulky_items , has_fragile_items , suggested_helpers , recommended_vehicle_type_id , fits_in_one_trip
GET /orders/{id}	same 9 as POST /bookings

Three things not to miss

1. volume_cft is **not** length × width × height.
2. Always send booleans explicitly — a missing boolean reads as false in the app.
3. fits_in_one_trip: false means someone should call the customer.